LIBRARY MANAGEMENT SYSTEM

Mini Project — Implementation Plan

System Design Laboratory | Problem Statement 3

Four-member implementation plan covering the complete application, all eight system-design concepts, integration, and performance evaluation.

Team allocation

Member	Primary ownership	Main deliverables
Member 1	Frontend + core backend	UI, authentication, book/member CRUD, borrowing, returns, reservations, fines/payments, unique IDs, API integration
Member 2	Book discovery	Trie autocomplete, Redis key-value/cache, book search, external/simulated catalogue crawler
Member 3	Distributed storage	MongoDB sharding, consistent-hashing router, node distribution and scalability demonstration
Member 4	API protection + sharing + evaluation	Token Bucket rate limiter, book URL shortener, JMeter/Postman tests, metrics and graphs

Guiding principle: Each member owns a module, but all modules must be integrated into one Library Management System. The eight concepts must support real library workflows, not exist as unrelated demonstrations.

1. Project Scope

Build a prototype that supports member registration and authentication, book catalogue browsing and search, availability checking, borrowing and returning, reservations, and fine/payment management. The system should remain responsive as books, members, transactions, and concurrent requests increase.

Core user flows

- **Member flow:** Register/login → browse/search books → view availability → borrow or reserve → return → view any fine/payment status.
- **Catalogue flow:** Add/import book metadata → generate unique book ID → store in the database → expose search and autocomplete → cache frequent results.
- **Sharing flow:** Open a book detail page → request a short link → resolve the short link to the correct book page.
- **Protection flow:** Incoming API request → rate limiter checks the client bucket → request is allowed or rejected with HTTP 429.
- **Scale flow:** Book/member key is routed to a storage node → data is stored/read through the distributed-storage layer → distribution is measured.

2. Recommended Technology Stack

Layer	Suggested technology	Purpose
Frontend	HTML, CSS, JavaScript (or React if already familiar)	Member UI, catalogue, borrowing/reservations, staff/admin screens
Backend	Java + Spring Boot REST APIs	Business logic and integration of all modules
Primary database	MongoDB	Books, members, loans, reservations, fines and short-link mappings
Key-value/cache	Redis	Cached book details, availability snapshots, autocomplete results and rate-limit state
Infrastructure	Docker Compose	Run Redis and MongoDB nodes/services consistently
Testing	Postman + Apache JMeter	API correctness, concurrent load and performance measurements
Version control	Git + GitHub	Shared code, branches, integration and issue tracking

Use one shared repository and agree on API contracts, data models, package names, ports, and environment variables before parallel development begins.

3. Work Allocation — Combined Responsibilities

Member 1 — Frontend and Core Backend

- **Frontend:** Build the shared navigation/layout and pages for login/register, catalogue, book details, search results, member dashboard, borrowing/returns, reservations, fines/payments, and basic admin book/member management.
- **Backend:** Create the Spring Boot project structure, shared API response/error conventions, authentication flow, core MongoDB models/repositories, and REST endpoints for books, members, loans, returns, reservations, and fines.
- **Unique ID Generator:** Generate unique IDs for books, members, and loan/transaction records. Ensure IDs are checked or generated by a collision-resistant strategy; record collision count during tests.
- **Integration:** Connect the frontend to the REST APIs and coordinate the shared API contract with Members 2–4.

Member 2 — Search, Autocomplete, Redis and Crawler

- **Book search:** Search by title, author, ISBN and category, with pagination and availability filters.
- **Autocomplete:** Build a Trie over book titles/authors/categories; return the top-k suggestions for a typed prefix. Rebuild or update the Trie when catalogue data changes.
- **Key-Value Store:** Use Redis keys such as `book:{bookId}` and `search:{normalizedQuery}`. Use a cache-aside pattern: Redis hit returns cached data; miss queries MongoDB and then populates Redis. Invalidate/update relevant cache entries after book or availability changes.
- **Web Crawler:** Crawl only permitted or simulated catalogue pages, extract title/author/ISBN/description/rating where available, validate URLs, avoid duplicate URLs, and present imported metadata for review before adding it to the library catalogue. Do not rely on external sources for authoritative live stock counts.

Member 3 — Sharding and Consistent Hashing

- **Sharding:** Configure a MongoDB sharded deployment with config server, shard replica sets, and mongos router. Choose and document shard keys for book/member data; verify distribution with MongoDB commands.
- **Consistent hashing:** Implement a hash ring with virtual nodes to route catalogue keys to logical storage/application nodes. Keep this routing responsibility distinct from MongoDB's own shard routing; document how the application router and database sharding cooperate.
- **Scale demonstration:** Capture distribution before and after adding/removing a node. Measure how many keys move and the load per node. If a full production-grade multi-node setup is too costly, use a clearly labelled local Docker prototype and document limitations.

Member 4 — Rate Limiter, URL Shortener and Evaluation

- **Rate limiter:** Implement the Token Bucket algorithm, preferably with atomic Redis updates. Configure bucket capacity/refill rate per client or endpoint; return HTTP 429 when empty and record allowed/rejected requests.
- **URL shortener:** Create a short code for a book detail page and persist the mapping. Provide an endpoint to resolve the code and redirect or return the canonical book URL. Handle unknown codes safely.
- **Performance:** Build Postman test collections and JMeter plans; test rate limiting, cache hit/miss, autocomplete, short-link redirection, and the complete library workflow. Produce tables and graphs from measured results.

4. Eight Concepts — Integration Design

Concept	Where it appears in the system	Evidence to demonstrate	Owner
Rate Limiter	Protect login/search/borrow APIs from excessive requests	Allowed vs rejected requests; HTTP 429; request rejection rate	M4
Sharding	Partition book/member records across MongoDB shards	Shard status and per-shard record distribution	M3
Consistent Hashing	Route catalogue keys to logical nodes	Key-to-node mapping and keys moved after node addition/removal	M3
Key-Value Store	Redis caches popular book details and search suggestions	Cache hit/miss counts and lookup latency with/without cache	M2
Unique ID Generator	Book, member and transaction identifiers	Generation rate and collision count	M1
URL Shortener	Shareable links for book detail pages	Successful redirects, invalid-code handling, redirection latency	M4
Web Crawler	Import metadata from simulated/permitted catalogue pages	Pages crawled, valid records extracted, duplicates skipped, crawl throughput	M2
Autocomplete	Prefix suggestions for book title, author and category	Suggestion latency and accuracy for a fixed test set	M2

5. Suggested Data Model

Collection	Important fields	Notes
books	bookId, title, authors, ISBN, category, description, totalCopies, availableCopies, location, metadataSource	Use bookId/ISBN with appropriate uniqueness rules; availability changes on borrow/return.
members	memberId, name, email, passwordHash, role, createdAt	Never store plain-text passwords; restrict staff/admin operations.
loans	loanId, memberId, bookId, borrowedAt, dueAt, returnedAt, status	Use consistent updates to avoid lending more copies than available.
reservations	reservationId, memberId, bookId, createdAt, status, queuePosition	Define queue order and cancellation/fulfilment rules.
fines/payments	transactionId, loanId, memberId, amount, status, createdAt	Use simulated payments only unless a payment provider is explicitly required.
shortLinks	code, targetPath, createdAt, createdBy	Unique short code; reject unknown or unsafe redirect targets.

6. API Contract — Agree Before Coding

These are suggested API contracts. The team should finalize names and payloads together before members implement their modules.

Method + endpoint	Purpose / owner
POST /api/auth/register POST /api/auth/login	Register and authenticate members — Member 1
GET /api/books GET /api/books/{bookId} POST /api/books	Browse, view and add books — Member 1; search behavior coordinated with Member 2
GET /api/search?q=... GET /api/search/suggest?prefix=...	Search and Trie autocomplete — Member 2
POST /api/loans POST /api/loans/{loanId}/return	Borrow and return a book; validate availability — Member 1
POST /api/reservations GET /api/members/{memberId}/reservations	Create/list reservations — Member 1
GET /api/members/{memberId}/fines POST /api/fines/{id}/pay	Fine calculation/status and simulated payment — Member 1
POST /api/crawler/run?seedUrl=...	Start a controlled catalogue crawl — Member 2
POST /api/short-links GET /s/{code}	Create/resolve book share links — Member 4
GET /api/admin/rate-limit/stats	Show allowed/rejected request metrics — Member 4
GET /api/admin/distribution	Expose or document shard/hash-ring distribution for demonstration — Member 3

7. Integration Rules

- Keep one shared repository with a main branch and short-lived feature branches: `feature/core-ui`, `feature/search-cache`, `feature/distributed-storage`, and `feature/rate-shortener-tests`.
- Agree on port numbers, MongoDB/Redis configuration, JSON field names, ID formats, error response format, and CORS settings early.
- Use environment variables for connection strings and secrets; commit an `.env.example`, never real credentials.
- Define shared DTOs and error handling before integrating. Keep controller/service/repository responsibilities separate.
- Availability and loan creation must be consistent: two concurrent requests must not borrow the final copy twice. Add a concurrency test.
- Cache invalidation is mandatory when book metadata or availability changes; stale cached availability can otherwise mislead users.
- Use synthetic test data. Crawler targets must be permitted or simulated; do not treat crawled availability as the library's authoritative stock.

8. Implementation Timeline

Phase	Member 1	Member 2	Member 3	Member 4	Exit condition
1. Contract & setup	Create app skeleton, models and initial UI	Define search DTOs and sample catalogue	Design shard/hash-ring plan	Define rate-limit and short-link APIs; test plan	All members can run project and agree on contracts
2. Core modules	Login, CRUD, borrowing, returns, reservations, fines, UI	Trie, Redis cache, crawler	Sharded cluster and consistent-hash routing	Token Bucket and URL shortener	Each module passes isolated API tests
3. Integration	Connect all UI screens to APIs	Expose search/autocomplete/crawler APIs	Connect storage routing and prove read/write	Protect APIs and connect short-link flow	End-to-end flows work in one environment
4. Testing & report	Fix functional bugs and capture screenshots	Measure cache and autocomplete metrics	Measure shard distribution and node changes	Run JMeter, compile metrics/graphs	Evidence, graphs, architecture and demo ready

9. Testing and Performance Evaluation

Run at least three workload levels. The exact concurrency and request counts should be chosen based on available hardware and recorded in the report; the values below are a starting template, not measured results.

Workload	Suggested concurrent users	Record
Low	10	Average response time, throughput, errors, CPU/memory if available
Medium	50	Same metrics; compare with low workload
High	100 or a safe machine-specific level	Same metrics; identify saturation and bottlenecks

Record these component-specific measurements

- Rate limiter: allowed requests, rejected requests, rejection rate, and response behavior at capacity.
- Redis: cache hit ratio and lookup latency compared with direct database reads.
- Autocomplete: response time and accuracy against a fixed set of expected suggestions.
- Crawler: pages visited per second, valid records extracted, and duplicate URLs skipped.
- Unique IDs: IDs generated per second and collision count.
- URL shortener: successful/failed resolutions and redirection latency.
- Sharding/consistent hashing: records per shard/node, requests per node, and keys remapped after node addition/removal.

Do not invent or estimate results in the final report. Capture actual outputs from JMeter, logs, MongoDB commands, and Redis metrics.

10. Scalability and Failure Demonstration

- **Data growth:** Load a larger synthetic book/member dataset and compare search latency and database response time.
- **Cache comparison:** Compare repeated search with Redis enabled versus a direct database baseline.
- **Rate-limit test:** Send a burst of requests from one client; show permitted requests, HTTP 429 responses, and recovery after token refill.
- **Node addition:** Add a logical consistent-hash node and record the number/proportion of keys remapped. If the MongoDB shard topology is changed, separately report MongoDB's own balancing behavior.
- **Failure test:** Stop a non-critical test node or make a test endpoint unavailable and record errors/recovery. Do not claim high availability unless it is actually configured and verified.
- **Consistency test:** Simulate concurrent borrow requests for the final copy and verify that only one succeeds.

11. Final Deliverables Checklist

- Requirement analysis and justification for all eight concepts.
- Architecture diagram showing frontend, REST API, Redis, MongoDB sharded cluster, crawler and request flow.
- Working frontend and backend integrated into one application.
- Demonstration of registration/login, book search/autocomplete, availability, borrow/return, reservation and fine status.
- Demonstration of all eight concepts through real system workflows.
- Postman collection, JMeter test plan, actual measurement tables and graphs.
- Scalability/failure analysis, observed bottlenecks, trade-offs and limitations.
- README with setup instructions, Docker Compose commands, API documentation and team contributions.

12. Suggested Live Demonstration Sequence

Step	Demonstration	Concepts shown
1	Register/login and create a book/member record	Unique ID Generator, core system
2	Type a title prefix and show suggestions; search a book twice	Autocomplete, Key-Value Store
3	Run crawler against simulated catalogue pages and inspect extracted metadata	Web Crawler
4	Borrow/return/reserve and view availability/fine status	Core workflows and consistency
5	Create a short link and open it; then burst requests until limited	URL Shortener, Rate Limiter
6	Show shard/node distribution and add a logical node	Sharding, Consistent Hashing
7	Show low/medium/high load results and discuss bottlenecks	Performance evaluation

Definition of done: All eight concepts are implemented, connected to the Library Management System, demonstrated with observable evidence, and evaluated with actual measurements. The system should be presented as a prototype, with limitations stated honestly.